Prácticas

Programación de sockets

Redes y Sistemas Distribuidos

Grado en Ingeniería Informática

Curso 2025-2026

Universidad de La Laguna

Control de versiones

Curso:	2025-2026
Version:	0.5.5
Git Hash:	53a00a8e13312a6cbfcc9987dc560a68182595c4
Fecha:	2026-04-20 10:36:23 +0100

Contents

1 Introduction **1**

2 Outcome of this assignment: Gopher and Gemini Protocol Servers **1**

3 Why Gopher and Gemini? **1**

4 The Gopher Protocol (RFC 1436) **1**
- 4.1 Protocol Overview . . . 1
- 4.2 Request Format . . . 2
- 4.3 Response Format . . . 2
- 4.4 Gopher Item Types . . . 2

5 The Gemini Protocol **2**
- 5.1 Protocol Overview . . . 3
- 5.2 Request Format . . . 3
- 5.3 Response Format . . . 3
- 5.4 Gemini Status Codes . . . 4
- 5.5 Gemtext Format . . . 4

6 Documentation about system calls related with sockets **4**

7 Code template **4**

8 Functions to implement **5**
- 8.1 Level 1: Socket Operations . . . 5
- 8.2 Level 2-3: Gopher Protocol . . . 5
- 8.3 Level 4-5: Gemini Protocol . . . 5
- 8.4 Level 6: TLS (Advanced) . . . 5

9 Common issues **6**

10 Debugging and testing **6**
- 10.1 Compilation . . . 6
- 10.2 Running the servers . . . 6
- 10.3 Manual testing with `netcat` . . . 6
- 10.4 Manual testing with clients . . . 7
  - 10.4.1 Testing Gopher with **Lynx** . . . 8
- 10.5 Testing Gemini with **Amfora** . . . 10
- 10.6 Test levels . . . 12
- 10.7 Example: Testing Gopher file request . . . 13
- 10.8 Example: Testing Gopher directory listing . . . 13
- 10.9 Example: Testing Gemini file request . . . 13
- 10.10 Example: Testing Gemini directory listing . . . 13

11 Additional resources **14**

1 Introduction

After reviewing the video lectures and the examples about socket programming that are available at the Virtual Campus of this course, the students should be able to develop their own application protocol for a simple client-server application.

2 Outcome of this assignment: Gopher and Gemini Protocol Servers

The goal of this assignment is to practice the implementation of two lightweight internet protocols: **Gopher** (RFC 1436) and **Gemini**. Students will implement both protocol servers in a single codebase, learning about protocol design, socket programming, and the evolution of internet protocols from the early 1990s to modern lightweight alternatives.

3 Why Gopher and Gemini?

These protocols were chosen for their educational value:

- **Simplicity**: Both protocols are much simpler than HTTP, making them ideal for learning socket programming and protocol implementation.
- **Progressive complexity**: Gopher is extremely simple (request-response), while Gemini adds status codes and MIME types, providing incremental learning.
- **Historical perspective**: Gopher (1991) represents early internet protocols, while Gemini (2019) shows modern minimalist design.
- **Security awareness**: Students learn about path traversal attacks, input validation, and TLS implementation.

4 The Gopher Protocol (RFC 1436)

Gopher is a distributed document search and retrieval protocol designed in 1991 at the University of Minnesota. It predates the World Wide Web and uses a simple menu-based interface.

4.1 Protocol Overview

Gopher uses a simple request-response model over TCP, typically on port 7070 (or 70 for production servers):

1. Client connects to server
2. Client sends a selector string followed by CR+LF (`\r\n`)
3. Server responds with either:

- File contents (for files)
- Menu listing (for directories)

4. Connection closes

4.2 Request Format

A Gopher request consists of a selector string:

```
<selector><CR><LF>
```

Examples:

- Root directory: `/\r\n` or just `\r\n`
- Specific file: `/docs/readme.txt\r\n`
- Subdirectory: `/images/\r\n`

4.3 Response Format

For files: The server sends the file contents directly and closes the connection.

For directories: The server sends a menu in tab-delimited format:

```
<type><display_text><TAB><selector><TAB><host><TAB><port><CR><LF>
<type><display_text><TAB><selector><TAB><host><TAB><port><CR><LF>
...
.<CR><LF>
```

Example menu:

```
0README.txt        /README.txt        localhost        7070
1Documents        /docs        localhost        7070
9binary.zip        /files/binary.zip        localhost        7070
.
```

4.4 Gopher Item Types

5 The Gemini Protocol

Gemini is a modern protocol designed in 2019 as a lightweight alternative to HTTP. It's heavier than Gopher but much simpler than HTTP, providing a middle ground for privacy-focused, lightweight content delivery.

Type	Description
0	Text file (.txt, .md)
1	Directory/Menu
3	Error message
9	Binary file (.zip, .exe, .bin)
g	GIF image
I	Image file (other formats)
h	HTML file

Table 1: Gopher item types to implement

5.1 Protocol Overview

Gemini uses a URL-based request system over TCP, typically on port 1965:

1. Client connects to server (optionally with TLS)
2. Client sends a URL followed by CR+LF
3. Server responds with a status line and optional body
4. Connection closes

5.2 Request Format

A Gemini request is a URL (maximum 1024 bytes):

```
<URL><CR><LF>
```

Examples:

- `gemini://localhost/\r\n`
- `gemini://example.com/docs/file.gmi\r\n`
- `gemini://example.com:1965/index.gmi\r\n`

5.3 Response Format

All responses start with a status line:

```
<STATUS><SPACE><META><CR><LF>
<BODY>
```

Examples:
Success:

```
20 text/gemini
# Welcome to Gemini
=> /docs/ Documentation
```

Not found:

```
51 Not found
```

5.4 Gemini Status Codes

Code	Meaning
20	SUCCESS - Request successful
40	TEMPORARY FAILURE - Generic error
51	NOT FOUND - Resource not found
59	BAD REQUEST - Malformed request

Table 2: Gemini status codes to implement

For status 20 (success), the meta field contains the MIME type (e.g., `text/plain`, `text/gemini`, `image/jpeg`).

5.5 Gemtext Format

Gemini's native hypertext format is called "gemtext". For directory listings:

```
# Directory Listing

=> /file1.txt file1.txt
=> /file2.gmi file2.gmi
=> /subdir/ subdir/
```

6 Documentation about system calls related with sockets

It is recommended to study the Linux System Calls related to sockets. The related Linux `man` pages have enough documentation about these functions. The resources and videos available at the Virtual Campus are also useful. You should study at least the following system calls: `socket, bind, connect, listen, accept, send, recv, shutdown, close`.

For the TLS implementation (Gemini advanced feature), you should also study OpenSSL functions: `SSL_CTX_new, SSL_new, SSL_set_fd, SSL_accept, SSL_read, SSL_write`.

7 Code template

To facilitate the implementation, this assignment provides a code template that can be used. This template is composed of the following files:

- `common.h`: Common utility functions (MIME types, Gopher types, path validation)
- `protocol_server.cpp`: Main program with command-line parsing
- `ProtocolServer.h/cpp`: Base server class with TCP socket functionality
- `GopherServer.h/cpp`: Gopher protocol server implementation

- `GopherConnection.h/cpp`: Gopher connection handler (per-client)
- `GeminiServer.h/cpp`: Gemini protocol server implementation
- `GeminiConnection.h/cpp`: Gemini connection handler (per-client)
- `CMakeLists.txt`: Build configuration

Check the `TODO` comments in the files to know where you have to implement functionality. Most of the modifications have to be done in `ProtocolServer.cpp`, `GopherConnection.cpp`, and `GeminiConnection.cpp`.

8 Functions to implement

Students must implement the following functions (marked with TODO in the code):

8.1 Level 1: Socket Operations

- `define_socket_TCP()` - Create, bind, and listen on a TCP socket

8.2 Level 2-3: Gopher Protocol

- `read_selector()` - Read selector from client (until `\r\n`)
- `selector_to_path()` - Convert selector to filesystem path
- `send_file()` - Send file contents to client
- `send_directory()` - Generate and send Gopher menu

8.3 Level 4-5: Gemini Protocol

- `read_url()` - Read URL from client (max 1024 bytes)
- `parse_url_path()` - Extract path from `gemini://` URL
- `send_file()` - Send file with Gemini header (status + MIME type)
- `send_directory()` - Generate gemtext directory listing

8.4 Level 6: TLS (Advanced)

- Initialize TLS context with OpenSSL
- Wrap socket with `SSL_accept()`
- Use `SSL_read()` and `SSL_write()` for encrypted communication

9 Common issues

During the development process you may experience some common issues:

- **Line endings**: Both protocols require `\r\n` (CR+LF), not just `\n`
- **Path traversal**: Always validate paths to prevent `../` attacks using the provided `is_safe_path()` function
- **Buffer overflows**: Check input lengths (Gopher selector ¡ 1024, Gemini URL $\leq$ 1024)
- **Binary files**: Handle binary files correctly (don't add line endings)
- **Socket cleanup**: Always close sockets and file descriptors
- **Thread safety**: Be careful with shared resources in multi-threaded code
- **Implementation order**: Implement Gopher first (simpler), then Gemini

10 Debugging and testing

10.1 Compilation

```
cd protocol_server_cpp
cmake .
make
```

10.2 Running the servers

Gopher server:

```
./protocol_server gopher 7070
```

Gemini server:

```
./protocol_server gemini 1965
./protocol_server gemini 1965 --tls  # With TLS
```

10.3 Manual testing with `netcat`

Testing Gopher:

```
# Request root directory
echo "/" | nc localhost 7070

# Request specific file
echo "/test.txt" | nc localhost 7070

# Request subdirectory
echo "/docs/" | nc localhost 7070
```

Testing Gemini:

```
# Request root
echo -e "gemini://localhost/\r\n" | nc localhost 1965

# Request specific file
echo -e "gemini://localhost/test.txt\r\n" | nc localhost 1965
```

10.4 Manual testing with clients

While `netcat` is excellent for debugging and understanding raw protocol communication, testing with real clients is essential for verifying complete protocol compliance. Real clients provide:

- **Visual feedback**: See how your server content is rendered in practice
- **Navigation**: Test directory browsing and link following
- **Strict protocol compliance**: Real clients enforce protocol specifications more rigorously than manual testing
- **User experience validation**: Understand how end-users will interact with your server

Protocol compliance is particularly important. A real Gopher or Gemini client will fail if your server implementation has subtle errors that may not be apparent when testing with `netcat`. This can happen when:

- The protocol specification was misunderstood during implementation
- Individual operations work correctly in isolation but fail when combined in realistic usage patterns

Therefore, `netcat` alone is insufficient to verify server functionality before submitting your implementation for automated testing and grading.

Many clients are available for both protocols. For Gopher, options include Lynx, Gopher, and Bombadillo (see wikipedia). For Gemini, popular clients include Amfora, Bombadillo, and others (see wikipedia).

This guide focuses on one client for each protocol and demonstrates a comprehensive testing workflow:

1. Connect to the server and list the root directory
2. Select and display a document
3. Navigate to a subdirectory
4. Display another document
5. Navigate back to the previous directory
6. Download a binary file

10.4.1 Testing Gopher with Lynx

Lynx is a veteran text-based browser that supports Gopher, HTTP, and other protocols. Install it using the appropriate command for your operating system:

```
# Debian/Ubuntu
~$ sudo apt install lynx
# Fedora/RHEL
~$ sudo dnf install lynx
# Arch Linux
~$ sudo pacman -S lynx
```

Before testing, ensure your server binary has been compiled as described in the compilation section. Navigate to the demo directory provided with the template repository:

```
~$ cd demo/gopher_demo
```

Start the Gopher server:

```
~$ ../../protocol_server_cpp/protocol_server gopher
Gopher server listening on port xxxxx
```

From a separate terminal, connect the Lynx client to your server (see Figure 1):

```
$ lynx gopher://localhost:xxxx
```

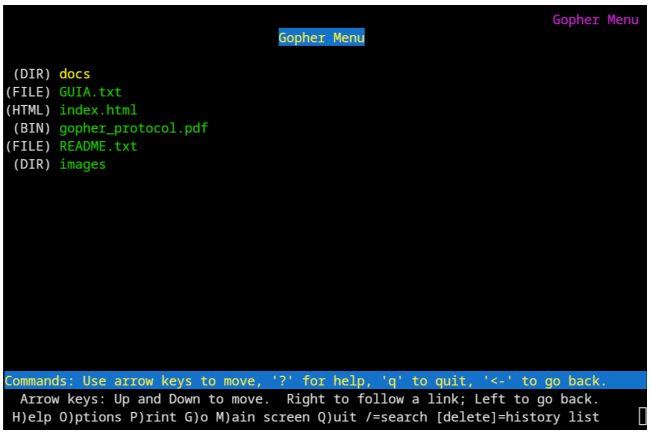

Figure 1: Lynx has connected to the server.

To view the contents of a text file, use the arrow keys to select it and press Enter (see Figure 2).

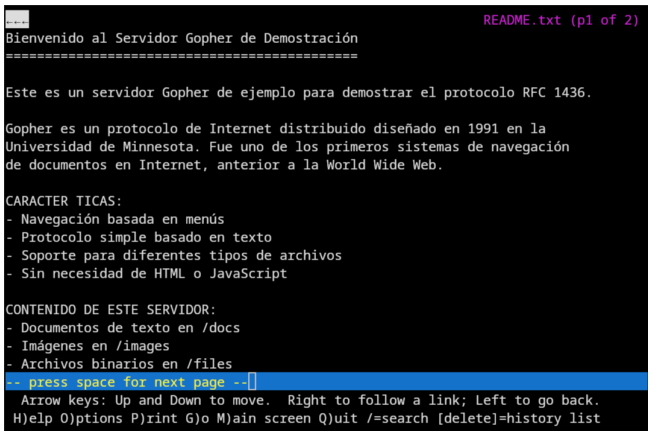

Figure 2: The file `README.txt` was selected.

Press the left arrow key to navigate back. To explore a subdirectory such as `/docs`, select it and press Enter. Lynx will retrieve and display its contents (see Figure 3).

Figure 3: Listing of the `/docs` sub-directory.

You can display the contents of any file in the same manner (see Figure 4).

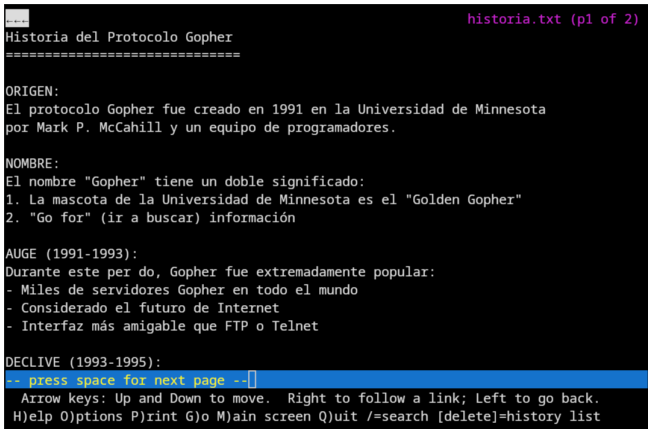

Figure 4: Listing of the `/docs` sub-directory.

Finally, navigate back to the root directory and select the PDF file. Lynx will present a download option (see Figure 5).

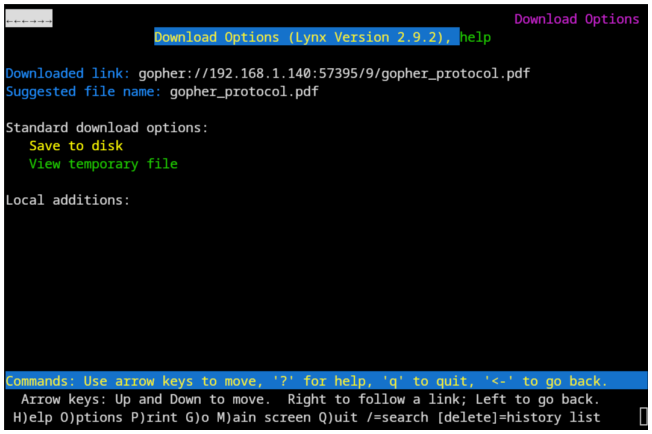

Figure 5: Downloading a file.

Verify that your implementation behaves correctly and that the file downloads successfully.

10.5 Testing Gemini with Amfora

Amfora is a feature-rich terminal client specifically designed for the Gemini protocol. Install it using one of the following methods:

```
# Download binary from releases
# https://github.com/makeworld-the-better-one/amfora/releases

# From source (requires Go)
~$ go install github.com/makeworld-the-better-one/amfora@latest

# Arch Linux
~$ yay -S amfora
```

Ensure your server binary has been compiled as described in the compilation section. Navigate to the Gemini demo directory:

```
~$ cd demo/gemini_demo
```

Generate TLS certificates for the server:

```
~$ openssl req -x509 -newkey rsa:4096 -keyout key.pem -out cert.pem -days 365 -nodes
```

Start the Gemini server with TLS enabled:

```
~$ ../../protocol_server_cpp/protocol_server gemini --tls
Gemini server listening  (TLS enabled) port = XXXX
```

Important: TLS is mandatory for testing with real Gemini clients, as it is a hard requirement in the Gemini protocol specification. You must have implemented TLS support before performing these tests.

From a separate terminal, launch the Amfora client:

```
~$ amfora gemini://localhost:34043
```

The client should display the directory listing of `/demo/gemini_demo` from the template repository (see Figure 6).

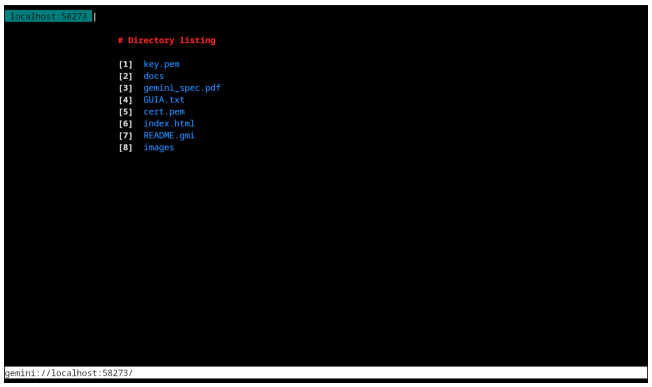

Figure 6: The amfora client connected with the gemini protocol and is listing the contents of the directory.

Navigate by entering the number corresponding to each item. For example, pressing `7` will display the `README.gmi` file. Since this is a gemtext file (the native document format for Gemini), Amfora will render it appropriately (see Figure 7).

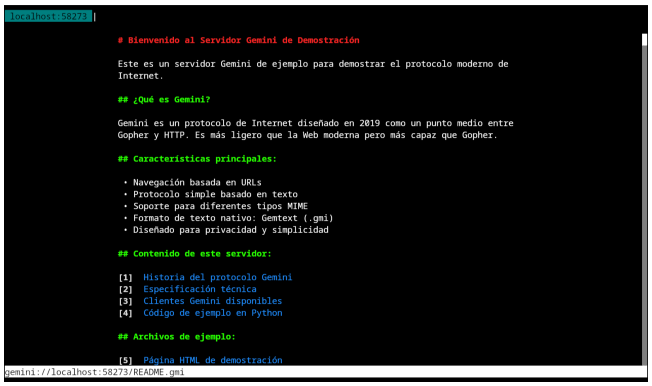

Figure 7: The `README.gmi` gemtext is shown.

Press the `b` key to return to the previous view (Figure 6). To navigate into a subdirectory, select it by number. For instance, pressing `2` will access `/docs/` and display its contents (see Figure 8).

Figure 8: The contents of the subdirectory `/docs/` are shown.

Pressing `1` will display the gemtext file `/docs/clients.gmi` (see Figure 9).

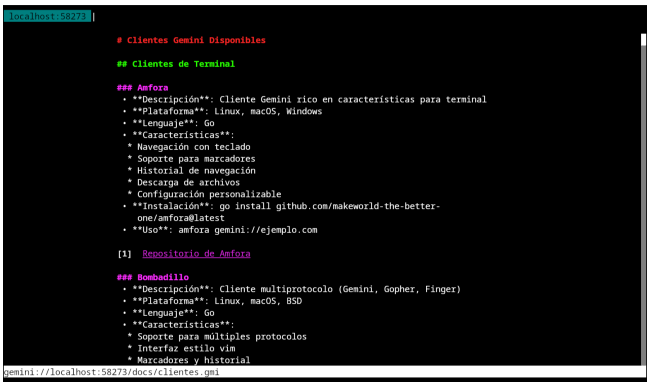

Figure 9: The contents the `/docs/clients.gmi` gemtext file are shown.

Finally, return to the root directory by pressing the `B` key twice, then press `3` to download the file `gemini_spec.pdf` (see Figure 10).

Figure 10: A pdf file can be downloaded.

Verify that your implementation behaves correctly and that the file downloads successfully.

10.6 Test levels

1. **Level 1** (3 tests): Basic socket operations

2. **Level 2** (4 tests): Gopher protocol basics
3. **Level 3** (5 tests): Gopher advanced features
4. **Level 4** (4 tests): Gemini protocol basics
5. **Level 5** (5 tests): Gemini advanced features
6. **Level 6** (3 tests): Gemini TLS (advanced)
7. **Level 7** (2 tests): Concurrent operations
8. **Level 8** (3 tests): Stress testing

10.7 Example: Testing Gopher file request

```
user@localhost:~$ ./protocol_server gopher 7070
Gopher server listening on port 7070

# In another terminal:
user@localhost:~$ echo "/test.txt" | nc localhost 7070
Hello, World!
This is a test file.
```

10.8 Example: Testing Gopher directory listing

```
user@localhost:~$ echo "/" | nc localhost 7070
0test.txt        /test.txt        localhost        7070
1docs        /docs        localhost        7070
9binary.bin        /binary.bin        localhost        7070
.
```

10.9 Example: Testing Gemini file request

```
user@localhost:~$ ./protocol_server gemini 1965
Gemini server listening on port 1965 (TLS disabled - for testing only)

# In another terminal:
user@localhost:~$ echo -e "gemini://localhost/test.txt\r\n" | nc localhost 1965
20 text/plain
Hello, World!
This is a test file.
```

10.10 Example: Testing Gemini directory listing

```
user@localhost:~$ echo -e "gemini://localhost/\r\n" | nc localhost 1965
20 text/gemini
# Directory Listing
```

```
=> /test.txt test.txt
=> /index.gmi index.gmi
=> /docs/ docs/
```

11 Additional resources

- RFC 1436 - The Internet Gopher Protocol: https://www.rfc-editor.org/rfc/rfc1436
- Gemini Protocol Specification: https://gemini.circumlunar.space/docs/specification.gmi
- Gopher Wikipedia: https://en.wikipedia.org/wiki/Gopher_(protocol)
- Project README: Detailed implementation guide in the repository
- PROTOCOLS.md: Complete protocol specifications with examples